Reviewer Pack — Tensor Rank Exponential Gap in Read-Twice vs Read-Once Branc…

Entry ec8a0368610a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 00:53:25 UTC

Tensor Rank Exponential Gap in Read-Twice vs Read-Once Branching Programs for IP_2

Entry ID: `ec8a0368610a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-10 00:53:25 UTC

1. Conjecture

Field A (mathematical branch): Tensor Rank over Real Numbers **Field B** (complexity object): Read-Twice Branching Programs

Statement:

> For any read-once branching program P over n variables, the tensor rank of the transition tensor T(P) is O(log n). For the IP_2 function, any read-twice branching program Q must satisfy rank(T(Q)) ≥ 2^{n/2} / poly(n).

Rationale (proposer's reasoning):

> Tensor rank captures the algebraic complexity of branching program transitions. Read-once BPs decompose as low-rank tensors, while read-twice BPs require high-rank tensors to model their repeated access to variables, mirroring the exponential gap in communication complexity for IP_2.

Taxonomy category: `NISAN_WIGDERSON_DESIGNS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `27e0be744337b491`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def matrix_multiplication(A, B):
    rows_A = len(A)
    cols_A = len(A[0])
    rows_B = len(B)
    cols_B = len(B[0])

    if cols_A != rows_B:
        raise ValueError("Matrix dimensions must be compatible for multiplication")

    result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]

    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                result[i][j] += A[i][k] * B[k][j]

    return result

def lu_decomposition(A):
    n = len(A)
    L = [[0 for _ in range(n)] for _ in range(n)]
    U = [[0 for _ in range(n)] for _ in range(n)]

    for i in range(n):
        L[i][i] = 1
        for j in range(i, n):
            sum_upper = sum(L[k][j] * U[k][i] for k in range(i))
            U[i][j] = A[i][j] - sum_upper
        for k in range(i + 1, n):
            sum_lower = sum(L[k][i] * U[i][j] for j in range(i))
            L[k][i] = (A[k][i] - sum_lower) / U[i][i]

    return L, U

def tensor_rank(matrix):
```

```
    n = len(matrix)
    if n == 0:
        return 0
    if n == 1:
        return 1

    L, U = lu_decomposition(matrix)
    rank = sum(1 for row in U if any(val != Fraction(0) for val in row))
    return rank

def generate_read_once_bp(n):
    bp = []
    for _ in range(n):
        layer = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
        bp.append(layer)
    return bp

def generate_read_twice_bp(n):
    bp = []
    for _ in range(2):
        layer = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
        bp.append(layer)
    return bp

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    read_once_rank_sum = 0
    read_twice_rank_sum = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5):  # Ensure at least 5 instances per seed
            read_once_bp = generate_read_once_bp(n)
            read_twice_bp = generate_read_twice_bp(n)

            read_once_matrix = matrix_multiplication(*read_once_bp)
            read_twice_matrix = matrix_multiplication(*read_twice_bp)

            read_once_rank = tensor_rank(read_once_matrix)
            read_twice_rank = tensor_rank(read_twice_matrix)

            if read_once_rank > 4 * math.log(n):
                return {
                    "metric_name": "Read-Once Rank",
                    "metric_value": read_once_rank,
                    "instances_tested": instances_tested,
                    "conjecture_holds": False,
                    "counterexample": f"n={n}, rank={read_once_rank} > 4 * log({n})"
                }

            if read_twice_rank < 2**(n/2) / n**2:
                return {
                    "metric_name": "Read-Twice Rank",
                    "metric_value": read_twice_rank,
```

```
                    "instances_tested": instances_tested,
                    "conjecture_holds": False,
                    "counterexample": f"n={n}, rank={read_twice_rank} < 2^{n/2} / {n**2}"
                }

            read_once_rank_sum += read_once_rank
            read_twice_rank_sum += read_twice_rank
            instances_tested += 1

    mean_read_once_rank = read_once_rank_sum / instances_tested
    mean_read_twice_rank = read_twice_rank_sum / instances_tested

    return {
        "metric_name": "Read-Once Rank",
        "metric_value": mean_read_once_rank,
        "instances_tested": instances_tested,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 7 for i in range(5, 8)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_

    mean_read_once_rank = sum(result['metric_value'] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result['conjecture_holds']) / len(results)

    if all(result['conjecture_holds'] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_read_once_rank} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_read_once_rank} std=0.0 support_fraction={support_fra
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6ec1712e.py", line 135, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6ec1712e.py", line 90, in run_trial
    read_once_matrix = matrix_multiplication(*read_once_bp)
                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: matrix_multiplication() takes 2 positional arguments but 5 were given
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed due to incorrect argument count in matrix multiplication, preventing result collection | next: Fix matrix_multiplication argument count in test_6ec1712e.py

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	50244
2	propose	ollama_remote	qwen3:8b	0	0	109217
3	preregistration	ollama_remote	qwen3:8b	0	0	24146
4	novelty	ollama_remote	qwen3:8b	0	0	20616
5	novelty	ollama_remote	qwen3:8b	0	0	16069
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14930
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14604
8	judge	ollama_remote	qwen3:8b	0	0	17266

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 267093 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/ec8a0368610a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ec8a0368610a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/ec8a0368610a.tar.gz` (if generated)